

Solving Finite Sequential Games

1. Overview

Definition

A **finite sequential game** is a game in which players move in a known order, the game has finitely many decision nodes and actions, and later players may observe some or all earlier moves. It is normally represented in **extensive form** as a game tree. **Backward induction** solves a finite game of perfect information by starting at the last decision nodes, choosing an optimal action there, replacing each solved node by its resulting payoff, and repeating toward the root.

Why it matters

Sequential decisions cannot be analyzed only by asking what looks attractive at the first move. A rational first mover predicts how every later player will respond. Backward induction makes that anticipation explicit and rules out plans that depend on irrational future behavior.

Where it is used in real systems

- protocol negotiation and API version adoption;
- security attacker-defender models;
- database lock, abort, and retry decisions;
- cloud-provider pricing followed by customer demand;
- staged auctions, bargaining, release timing, and market entry;
- algorithmic planning, minimax search, and finite-horizon control.

Why interviewers ask about it

It tests whether a candidate can read a game tree, distinguish an action from a complete strategy, reason from future consequences rather than greedily, handle ties, and justify an equilibrium rather than merely name an outcome.

2. Core Idea

Intuition and analogy

Plan a trip backward from a fixed arrival time: determine the last train you can take, then the bus that reaches that train, then when to leave home. A sequential-game solution works the same way. At each future decision node, assume the player controlling that node chooses the continuation giving that player the highest payoff.

Small example

```

Entrant
├─ Stay Out ..... (2, 5)
└─ Enter
   └─ Incumbent

```

└ Fight	(-1, -1)
└ Accommodate	(4, 3)

Payoffs are (Entrant, Incumbent).

Step-by-step

1. Begin at the incumbent's last decision.
2. Compare the incumbent's payoffs: -1 from Fight and 3 from Accommodate.
3. A rational incumbent accommodates.
4. Replace the Enter branch by its predicted result (4, 3).
5. At the root, the entrant compares 2 from Stay Out with 4 from Enter.
6. The entrant enters. The predicted path is Enter -> Accommodate.
7. A full strategy profile is (Enter; Accommodate after Enter), not merely the two-word outcome.

3. Important Subtopics

3.1 Extensive-form representation

Meaning: A rooted tree contains decision nodes, actions on edges, terminal histories, player labels, and payoff vectors. **Why it matters:** timing and information are visible, unlike in a bare payoff matrix. **Example:** a client sends or waits, then a server accepts or rejects. **Interview angle:** label whose payoff appears in each coordinate and who moves at every node.

3.2 Histories, terminal histories, and horizon

Meaning: A history is a sequence of actions from the root; a terminal history ends the game. A finite horizon guarantees that backward reasoning reaches the root. **Why it matters:** strategies condition on histories. **Example:** Enter, Accommodate is a terminal history. **Interview angle:** do not confuse a history (what happened) with a strategy (what would be done in every contingency).

3.3 Perfect information

Meaning: At every move, the acting player knows the complete earlier history. **Why it matters:** ordinary backward induction selects an action separately at each singleton information set. **Example:** chess has perfect information; simultaneous pricing does not. **Interview angle:** perfect information is not the same as complete information about payoffs or types.

3.4 Strategy versus action

Meaning: An action is one edge; a strategy specifies an action at every decision point belonging to that player, including unreached nodes. **Why it matters:** equilibrium evaluates unilateral changes to full plans. **Example:** "Reject after Low, accept after High" is a strategy. **Interview angle:** count pure strategies by multiplying action counts across that player's information sets.

3.5 Ties and multiple solutions

Meaning: A player may have several payoff-maximizing continuations. **Why it matters:** different tie choices can induce different earlier moves and multiple backward-induction equilibria. **Example:** a responder receives

2 from either Accept or Reject. **Interview angle:** branch on all best replies; do not silently choose the outcome convenient to the first mover.

3.6 Relation to dynamic programming and minimax

Meaning: Solving subtrees and propagating values upward is dynamic programming on a tree. In two-player zero-sum games, maximizing one's payoff and minimizing the opponent's yields minimax. **Why it matters:** it connects game theory to algorithms. **Example:** finite deterministic board-game search. **Interview angle:** general-sum nodes maximize the mover's own payoff coordinate, not a single globally shared value.

4. Real-World Example

A backend team decides whether to deploy a breaking API (**Deploy**) or keep the old API (**Keep**). After deployment, a client team chooses **Migrate** or **Refuse**.

History	Backend payoff	Client payoff	Interpretation
Keep	2	3	compatibility, maintenance cost
Deploy, Migrate	5	2	backend simplifies; client pays migration cost
Deploy, Refuse	-3	-2	outage

At the final node the client prefers **Migrate** ($2 > -2$). The backend therefore compares **Keep** $\rightarrow 2$ with **Deploy** $\rightarrow 5$ and deploys. The engineering lesson is practical: the prediction changes if migration is infeasible, payoffs include reputational damage, or the client cannot observe deployment. The tree's assumptions are part of the answer.

5. Diagrams / Mental Models

```

Enumerate tree -> Solve deepest decision nodes -> Collapse solved branches
      ^                                     |
      |                                     v
Verify every node <- Record full strategies <- Reach root and outcome

```

Object	Question to ask
Decision node	Who moves here?
Edge	What action is available?
Information set	What does the mover know?
Leaf	What is every player's payoff?
Continuation	Which leaf does this player prefer?

6. Common Interview Questions

1. **What is backward induction?** Solve the last decisions first and propagate rational continuation payoffs to the root. **Expected:** finite horizon, mover's payoff, recursive logic. **Mistake:** starting at the root

greedily.

2. **When is it directly applicable?** Finite extensive-form games with perfect information; ties are allowed but may create several solutions. **Expected:** assumptions. **Mistake:** claiming it directly solves every imperfect-information game.
3. **Why start at terminal-adjacent nodes?** No unresolved strategic consequence remains there, so the acting player's comparison is well defined. **Expected:** base case. **Mistake:** maximizing total welfare instead of the mover's payoff.
4. **What is returned: an outcome or strategies?** A complete contingent strategy profile plus its induced path and payoffs. **Expected:** off-path actions. **Mistake:** reporting only the observed path.
5. **How are ties handled?** Retain every maximizing action and propagate each possible continuation. **Expected:** multiplicity. **Mistake:** arbitrary tie-breaking without saying so.
6. **What is the running time on an explicit tree?** Linear in nodes or edges if every node is visited once, aside from representing multiple tied solutions. **Expected:** $O(|V| + |E|)$. **Mistake:** confusing tree size with compact game-description size.
7. **Is backward induction the same as greedy choice?** No. It uses solved future responses; a root-level greedy choice ignores them. **Expected:** future-contingent reasoning. **Mistake:** equating local immediate reward with continuation payoff.
8. **How is it related to minimax?** Minimax is the zero-sum specialization; general-sum backward induction maximizes the current mover's coordinate. **Expected:** zero-sum restriction. **Mistake:** minimizing the opponent in every general-sum game.
9. **Can the first mover always benefit from moving first?** No; commitment or information can help or hurt, and some games have a second-mover advantage. **Expected:** no universal rule. **Mistake:** assuming Stackelberg leadership always improves payoff.
10. **What breaks with an infinite horizon?** There may be no last node, so the basic base case disappears; fixed points, discounting, or repeated-game methods are needed. **Expected:** finite-horizon dependence. **Mistake:** applying the same mechanical procedure forever.

7. Deep-Dive Questions

1. **Why can a compactly described game still be hard?** The explicit tree may be exponential in horizon because every action creates branches; linear time in tree size can still be exponential in input size.
2. **What if future players are not perfectly rational?** Backward induction becomes a benchmark, not necessarily a behavioral prediction; bounded rationality, quantal response, or empirical probabilities can replace exact maximization.
3. **How do chance nodes change the calculation?** Compute expected continuation payoffs using the stated probabilities, then let the player choose the action with greatest expected payoff.
4. **Can backward induction solve imperfect-information games?** Not by independently optimizing every node in a non-singleton information set. Strategies must choose one action for the whole information set; methods such as sequential equilibrium may be needed.
5. **Why is centipede-game behavior surprising?** Backward induction predicts immediate stopping, yet humans often continue because of bounded rationality, social preferences, reputation, or uncertainty about types.

8. Comparison Tables

Backward induction	Forward/greedy reasoning
Starts at final decisions	Starts from current visible reward
Includes optimal continuation responses	May ignore later reactions
Produces contingent strategies	Often produces only a path
Correct for finite perfect-information solution	Can fail even in two-stage games
Sequential game	Simultaneous game
Order and observations matter	Players choose without observing current choices
Usually shown as a tree	Usually shown as a matrix
Backward induction may apply	Best-response/Nash analysis is standard

9. Common Mistakes

- Reading payoff coordinates in the wrong player order.
- Choosing the largest sum of payoffs rather than the mover's payoff.
- Reporting the equilibrium path but omitting off-path strategy choices.
- Ignoring ties or chance probabilities.
- Assuming later actions are observed when they are in the same information set.
- Calling every backward-induction outcome unique.
- Treating the tree's payoff numbers as immediate rewards rather than terminal utilities unless specified.

10. Edge Cases / Special Cases

- A one-node game reduces to choosing a payoff-maximizing action.
- With ties, the equilibrium set may be large and earlier incentives may depend on tie-breaking.
- Chance nodes are evaluated by expectation, not maximization.
- In imperfect-information games, nodes in one information set cannot receive different planned actions.
- In games with simultaneous moves inside stages, solve each terminal subgame's Nash equilibria, then reason backward; equilibrium selection may matter.
- Infinite or cyclic games need additional assumptions such as discounting, termination probabilities, or value-function fixed points.

11. How to Explain in Interview

"Backward induction solves a finite sequential game from the leaves upward. At each last unresolved decision node, I select the action maximizing the player-to-move's continuation payoff, collapse that subtree, and repeat to the root. I then report both the induced path and each player's complete contingent strategy. The direct method assumes perfect information; ties can produce multiple solutions."

12. Quick Revision Notes

- **Key definitions:** history = past actions; strategy = complete plan; terminal history = leaf; backward induction = leaf-to-root optimization.
- **Must remember:** maximize the current mover's payoff coordinate.
- **Comparison:** minimax is the zero-sum special case.

- **Trap:** outcome path is not a full strategy profile.
- **Complexity:** linear in the explicit tree, which may itself be exponentially large.

13. Practice Tasks

1. Draw and solve the entry-deterrence tree above; write full strategies.
2. Add a tie at the incumbent node and enumerate all solutions.
3. Write a recursive C++ function whose node stores **player**, children, and terminal payoff vector; return the selected payoff and action.
4. Add a chance node representing a 20% deployment failure and recompute expected values.
5. Convert a two-stage simultaneous-then-sequential story into a tree and identify why plain node-by-node maximization fails at the simultaneous stage.

14. Final Cheat Sheet

Item	Recall
Core definition	Solve optimal continuation play from leaves to root
Why it matters	Earlier choices depend on rational later responses
Most asked	assumptions, complete strategies, ties, complexity, minimax relation
Main comparison	backward induction uses continuation values; greedy reasoning does not
One-line answer	"Solve the last mover's best choice, collapse it, and repeat backward."

Credible and Non-Credible Threats

1. Overview

Definition

A **threat** is a promised harmful response intended to influence an earlier action. It is **credible** if carrying it out is optimal for the threatening player when the relevant decision point is actually reached. It is **non-credible** if the player would prefer not to carry it out once the time comes.

Threat credibility matters because Nash equilibrium can sometimes support an outcome using off-path behavior that no rational player would execute. Sequential rationality and SPNE remove such unsupported threats.

It appears in market-entry deterrence, rate-limit enforcement, security policy, contract penalties, negotiation, protocol sanctions, and platform moderation. Interviewers use it to test whether candidates separate announced policy from incentive-compatible behavior.

2. Core Idea

Consider the entry tree:

```

Entrant: Out -> (2,5)
        Enter
          Incumbent: Fight -> (-1,-1)
                    Accommodate -> (4,3)

```

The incumbent may announce "If you enter, I will fight." Before entry, this could scare the entrant. After entry, however, the incumbent compares **-1** with **3** and accommodates. The threat is non-credible. A credible threat must survive the question: **Would you still choose it after the history you hoped to prevent has occurred?**

3. Important Subtopics

3.1 Ex ante versus ex post incentives

Meaning: Ex ante, the threat may improve the threatener's outcome by changing another player's choice; ex post, execution may be costly. **Why:** credibility is evaluated at execution time. **Example:** permanently blocking a large customer may deter abuse but harm revenue after abuse occurs. **Interview:** compare payoffs at the later node, not at the root.

3.2 Empty threats and equilibrium support

Meaning: An empty threat specifies irrational behavior at an unreachable node. **Why:** it may make a Nash equilibrium look plausible in strategic form. **Example:** (Stay Out, Fight if Entry) can be Nash because the incumbent's node is never reached, yet Fight is suboptimal there. **Interview:** explain why Nash alone may not impose sequential rationality.

3.3 Commitments

Meaning: Commitment changes future feasible actions or payoffs so carrying out the response becomes optimal or automatic. **Why:** it can turn a non-credible statement into credible policy. **Example:** an automated quota service rejects requests beyond a published limit. **Interview:** credible commitment must be observable and hard or costly to reverse.

3.4 Threats versus promises

Meaning: A threat conditions punishment on an unwanted action; a promise conditions a reward on a desired action. Either may lack credibility. **Why:** both are contingent commitments. **Example:** "Stay under quota and receive priority" is a promise. **Interview:** test future incentives in the same way.

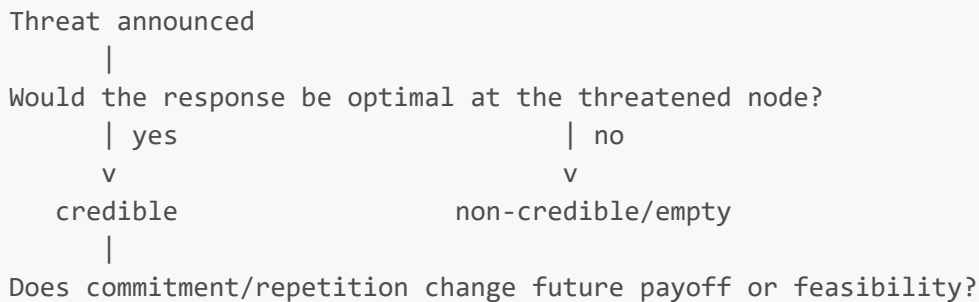
3.5 Reputation and repeated interaction

Meaning: A costly response today may protect future payoffs by establishing a reputation. **Why:** an action non-credible in a one-shot game can be rational in a repeated game. **Example:** a cloud provider compensates every SLA breach to preserve trust. **Interview:** include discounted future gains in payoffs; do not invoke reputation vaguely.

4. Real-World Example

A database gateway says it will permanently ban any client that exceeds a burst limit. If a high-value client bursts, permanent banning costs the gateway 10, while temporary throttling costs 2. Without automated or contractual commitment, permanent banning is non-credible. A token-bucket limiter that automatically rejects excess traffic is credible because enforcement is immediate and the gateway has chosen the mechanism beforehand. Good system design aligns the stated rule with what operators will rationally do during an incident.

5. Diagrams / Mental Models



Test	Credible	Non-credible
Optimal when reached?	Yes	No
Needs opponent to believe irrationality?	No	Usually
Survives backward induction?	Yes	No
Can support SPNE?	Yes	No

6. Common Interview Questions

1. **What makes a threat credible?** It is optimal to execute at the relevant future decision point. **Expected:** sequential incentive. **Mistake:** saying credibility means the statement sounds believable.
2. **Can a threat be costly and credible?** Yes, if every available alternative is worse or future benefits exceed the cost. **Expected:** relative payoffs. **Mistake:** equating cost with irrationality.
3. **Why can Nash equilibrium contain empty threats?** Unreached actions may not affect the threatener's equilibrium payoff, so changing only them may not be profitable at the root. **Expected:** off-path reasoning. **Mistake:** saying Nash checks every subgame.
4. **How does SPNE treat threats?** It requires Nash equilibrium in every subgame, eliminating behavior that is suboptimal in a proper subgame. **Expected:** sequential rationality. **Mistake:** claiming it removes every implausibility under imperfect information.
5. **How can commitment create credibility?** It removes alternatives or changes future payoffs. **Expected:** observable, enforceable mechanism. **Mistake:** treating a repeated announcement as commitment.
6. **Is an automated punishment always credible?** Only if automation really binds and cannot be cheaply overridden. **Expected:** feasibility and governance. **Mistake:** ignoring an admin bypass.
7. **Can reputation make a threat credible?** Yes in repeated or incomplete-information games if present punishment improves sufficiently valuable future outcomes. **Expected:** future payoff. **Mistake:** importing reputation into a stated one-shot game.

8. **Threat versus warning?** A threat describes the speaker's contingent action; a warning may merely predict an external consequence. **Expected:** control over response. **Mistake:** using the words interchangeably.
9. **How do you test credibility numerically?** At the response node, compare the threatener's payoff from executing with every alternative. **Expected:** correct coordinate. **Mistake:** comparing the target's payoffs.
10. **Can a dominated action be a credible threat?** A strictly dominated continuation action cannot be credible when its dominating alternative remains available. **Expected:** local feasible actions. **Mistake:** applying normal-form dominance without checking the continuation.

7. Deep-Dive Questions

1. **Can burning resources create strategic value?** Yes. Sunk investment can alter future incentives or signal resolve, although waste alone does not guarantee credibility.
2. **Why is observability necessary?** A hidden commitment cannot change the opponent's earlier choice because the opponent cannot condition beliefs on it.
3. **Does SPNE solve credibility with imperfect information?** Not fully: a game may have no proper subgame at a problematic information set. Sequential or perfect Bayesian equilibrium adds beliefs and sequential optimality.
4. **How does discounting affect repeated threats?** Future reputation benefits are multiplied by a discount factor; low patience can make present punishment no longer worthwhile.
5. **Can delegation help commitment?** Yes. Giving enforcement to an agent with different incentives or rigid rules can make a response credible, but introduces agency and override risks.

8. Comparison Tables

Threat	Promise	Commitment
Punishes undesired action	Rewards desired action	Makes a future response binding or optimal
Credibility tested after violation	Credibility tested after compliance	Credibility comes from altered choices/payoffs
May be empty	May be empty	Must be sufficiently irreversible/ costly to reverse

9. Common Mistakes

- Judging credibility from confidence, morality, or past words rather than payoffs.
- Comparing payoffs before the threat node instead of at it.
- Assuming a costly threat is automatically non-credible.
- Treating software automation as irreversible without checking override paths.
- Using reputation in a one-shot complete-information model without changing the game.
- Concluding that any Nash equilibrium is free of empty threats.

10. Edge Cases / Special Cases

- Tied continuation payoffs make execution weakly credible but prediction may be fragile.

- A threat can be credible only for some player types in an incomplete-information game.
- Randomized punishments can be credible when mixing makes the threatener indifferent.
- Legal, technical, or reputational constraints may change feasible actions rather than merely payoffs.
- A punishment may be credible collectively but not for the individual assigned to execute it, creating an enforcement problem.

11. How to Explain in Interview

"A threat is credible only if the player would rationally carry it out after the threatened history occurs. I test it at that future node using the threatener's continuation payoffs. Nash equilibrium may retain irrational off-path threats; backward induction or SPNE removes them in finite perfect-information games. Commitment can restore credibility by changing future options or incentives."

12. Quick Revision Notes

- Credibility is an **ex post incentive** test.
- Empty threats can support Nash but not SPNE when a proper subgame exposes them.
- Commitment must be observable and binding enough.
- Repetition can change payoffs and hence credibility.
- Interview trap: "costly" does not mean "non-credible."

13. Practice Tasks

1. Change Fight's incumbent payoff from **-1** to **4**; resolve the tree.
2. Model an API ban, temporary throttle, and warning as three response actions.
3. Add a setup cost for an automatic enforcement service and find when commitment pays.
4. Build a two-period reputation example with discount factor **delta**.
5. Identify a real policy whose manual override makes its threat weak.

14. Final Cheat Sheet

Item	Recall
Core definition	Credible means optimal to execute when reached
Why it matters	Announcements influence behavior only when incentives support them
Most asked	empty threats, Nash vs SPNE, commitment, reputation
Main comparison	words announce; commitment changes the game
One-line answer	"Re-evaluate the threat at its future node; if the threatener would back down, it is empty."

Subgames

1. Overview

Definition

A **subgame** is a portion of an extensive-form game that begins at a decision node that is a singleton information set, contains every successor of that node, and never cuts an information set. It must itself be a valid game. The entire game is always a subgame; any smaller one is a **proper subgame**.

Subgames isolate continuation problems. They matter in protocol phases, staged negotiations, retry workflows, and any system in which rational behavior must remain valid after different histories. Interviewers ask about them because the definition is precise and exposes common confusion about information sets.

2. Core Idea

Think of a subgame as a legal “save point”: starting there, players have all information required by that portion of the original game, and no information set is torn in half.

```

P1
├ Stop -> (2,2)
└ Continue
  └ P2      <-- singleton node; valid proper-subgame root
    ├── L -> (3,1)
    └── R -> (0,4)

```

The subtree rooted at P2 is a proper subgame. Solve it independently: P2 selects **R**, then P1 anticipates payoff 0 from Continue and selects Stop.

If two P2 nodes are connected by one information set, neither node alone may start a subgame because doing so would pretend P2 knows which node was reached.

3. Important Subtopics

3.1 Singleton information-set root

Meaning: the root must not share its information set with another node. **Why:** otherwise the continuation omits a state the mover cannot distinguish. **Example:** a responder who observed the offer has a singleton node. **Interview:** “a subtree” is not automatically a subgame.

3.2 Closure under successors

Meaning: once included, all descendants and terminal histories must be included. **Why:** removing an available continuation changes the game. **Example:** include both Accept and Reject branches. **Interview:** a hand-picked branch is not a subgame.

3.3 No cutting information sets

Meaning: if a subgame includes one node from an information set, it includes every node in that set. **Why:** players must retain their original information. **Example:** two indistinguishable server states must remain together. **Interview:** draw information-set ovals before counting subgames.

3.4 Proper versus whole subgame

Meaning: the whole tree is a subgame; a proper subgame excludes at least the original root. **Why:** SPNE includes Nash rationality for the whole game and every continuation. **Example:** a game may have no proper subgames. **Interview:** answer whether terminal nodes count according to the convention stated; standard strategic subgames start at decision nodes.

3.5 Continuation game

Meaning: the game remaining after a history. Not every continuation is formally a subgame under imperfect information. **Why:** terminology affects which refinements have force. **Example:** a history inside a non-singleton information set defines a conceptual continuation but not a proper subgame. **Interview:** distinguish casual “remaining game” from the formal definition.

4. Real-World Example

A distributed job first selects a region. After that choice is observed, the regional controller chooses **Run** or **Shed**. Each observed-region controller node can root a subgame. If the controller receives an anonymized job and cannot tell which region-routing history produced it, those nodes share an information set; separating one as a subgame would falsely give the controller information. This matters when modeling failover policies: an equilibrium policy must be optimal given what the controller actually observes.

5. Diagrams / Mental Models

```

Candidate node
|
+-- singleton information set? -- no --> not a subgame root
|
yes
+-- include every descendant? ----- no --> not a subgame
|
yes
+-- split any information set? ----- yes -> not a subgame
|
no
v
valid subgame
  
```

Region of tree	Subgame?	Reason
Entire game	Yes	Always
Subtree at observed decision node	Usually	Singleton root and closed descendants
One branch after an action	Not necessarily	May not start at a decision node
Subtree rooted inside linked information set	No	Root is not singleton

6. Common Interview Questions

1. **Define a subgame.** A valid subtree beginning at a singleton information set, containing all successors, and not cutting information sets. **Expected:** all three conditions. **Mistake:** “any subtree.”

2. **Is the whole game a subgame?** Yes. **Expected:** proper-subgame distinction. **Mistake:** saying only smaller games count.
3. **What is a proper subgame?** Any subgame strictly smaller than the whole game. **Expected:** excludes original root. **Mistake:** equating proper with equilibrium-reached.
4. **Why must the root be singleton?** Otherwise the player cannot know that the proposed subgame was reached rather than another node in the same information set. **Expected:** preserve information. **Mistake:** focusing only on tree geometry.
5. **Why include all successors?** A subgame must preserve every feasible continuation. **Expected:** valid independent game. **Mistake:** pruning inconvenient actions.
6. **Can information sets cross a subgame boundary?** No. **Expected:** no cutting. **Mistake:** including only the visually nearby node.
7. **Can a game have no proper subgames?** Yes, common with simultaneous moves represented using a non-singleton information set. **Expected:** whole game remains. **Mistake:** claiming every second-stage node starts one.
8. **How do subgames help find SPNE?** Find Nash equilibrium in terminal proper subgames and work backward, ensuring equilibrium in every subgame. **Expected:** continuation rationality. **Mistake:** checking only the equilibrium path.
9. **Are terminal nodes subgames?** Under the usual definition subgames start at decision nodes, so no strategic subgame starts at a terminal node; state the convention. **Expected:** precision. **Mistake:** confidently mixing conventions.
10. **Can two subgames overlap?** In a tree with perfect recall, proper subgames are typically disjoint or nested; crossing overlap would violate tree ancestry/information constraints. **Expected:** tree structure. **Mistake:** treating arbitrary node sets as subgames.

7. Deep-Dive Questions

1. **Why may SPNE equal Nash when there are no proper subgames?** The only subgame is the whole game, so the extra requirement adds nothing.
2. **How does imperfect information weaken SPNE?** Non-credible behavior inside an information set may escape subgame testing because no proper subgame begins there.
3. **What does perfect recall contribute?** Players never forget their earlier information or actions, supporting coherent behavioral strategies and well-behaved continuation analysis.
4. **Can a subgame begin after a chance move?** It can begin at a later singleton decision node; one must include the correct conditional structure and all descendants.
5. **How do nested subgames support algorithms?** Solve innermost subgames, replace them by equilibrium continuations, then solve their parents, like dynamic programming.

8. Comparison Tables

Subgame	Arbitrary subtree	Continuation game
Formal definition	Geometric fragment	Informal/general remaining interaction
Singleton root required	No	Not always
Cannot cut information set	May	May not be a formal subgame
Used directly by SPNE	Yes	Only if it qualifies

9. Common Mistakes

- Calling every node and descendants a subgame.
- Forgetting that the whole game counts.
- Counting nodes inside one information set as separate roots.
- Omitting descendant actions.
- Ignoring information-set boundaries when the diagram is visually complex.
- Confusing “off equilibrium path” with “outside every subgame.”

10. Edge Cases / Special Cases

- Simultaneous-move games often have no proper subgames.
- A singleton information set does not suffice if a descendant information set crosses the boundary.
- Nested subgames should be solved from the innermost outward.
- Different textbooks differ on degenerate terminal-node subgames; state the decision-node convention.
- In games with imperfect recall, standard equilibrium refinements need extra care.

11. How to Explain in Interview

“A subgame is a self-contained continuation of an extensive-form game. It starts at a singleton information set, includes all descendants, and cannot cut any information set. The entire game counts; a proper subgame is smaller. These conditions preserve the original choices and information, allowing us to test sequential rationality.”

12. Quick Revision Notes

- Root: singleton information set.
- Contents: every successor.
- Boundary: never split an information set.
- Whole game: always a subgame.
- Interview trap: subtree does not imply subgame.

13. Practice Tasks

1. Draw a three-stage perfect-information tree and mark every proper subgame.
2. Connect two nodes with an information set and recount.
3. Explain why a selected action branch is not necessarily a subgame.
4. Model an unobserved retry cause in a backend controller.
5. For each candidate root, apply the three-condition checklist aloud.

14. Final Cheat Sheet

Item	Recall
Core definition	Singleton-rooted, successor-closed, information-set-preserving subtree
Why it matters	Isolates valid continuation games
Most asked	root condition, no cutting, proper subgame, no-proper-subgame case

Item	Recall
Main comparison	every subgame is a subtree; not every subtree is a subgame
One-line answer	"Start at a singleton information set, take all descendants, and cut no information set."

Subgame Perfect Nash Equilibrium (SPNE)

1. Overview

Definition

A strategy profile is a **Subgame Perfect Nash Equilibrium** if its restriction to **every subgame**, including the whole game, is a Nash equilibrium. SPNE refines Nash equilibrium by requiring rational play after every possible history represented by a subgame, including histories that equilibrium play never reaches.

It matters because a strategic plan should remain stable when contingencies occur. Uses include protocol escalation, bargaining, market entry, staged pricing, security response, and distributed recovery. Interviewers ask about SPNE to test equilibrium definitions, off-path reasoning, and the relation among Nash equilibrium, backward induction, and credibility.

2. Core Idea

Nash asks: "Given the whole strategy profile, can anyone profit by changing strategy alone?" SPNE additionally asks the same stability question inside every valid continuation.

In the entry game, (Stay Out; Fight after Entry) can be a Nash equilibrium in strategic form: if entry never occurs, changing the incumbent's off-path action alone changes no realized payoff. But it is not SPNE because in the proper subgame after Entry, the incumbent prefers Accommodate. Backward induction gives (Enter; Accommodate), which is Nash in the whole game and in the continuation subgame.

3. Important Subtopics

3.1 Refinement of Nash equilibrium

Meaning: every SPNE is a Nash equilibrium, but some Nash equilibria are not SPNE. **Why:** refinement removes equilibria supported by non-credible continuation behavior. **Example:** entry deterrence. **Interview:** prove inclusion using the fact that the whole game is a subgame.

3.2 Sequential rationality

Meaning: prescribed play is optimal whenever a subgame is reached. **Why:** systems encounter failures and off-path states. **Example:** a retry policy remains optimal after an unexpected timeout. **Interview:** SPNE checks subgames, not literally every information set in imperfect-information games.

3.3 Backward induction result

Meaning: in finite perfect-information games, backward induction constructs an SPNE in pure strategies. **Why:** each chosen action is optimal in its continuation. **Example:** solve terminal decision nodes upward.

Interview: uniqueness requires strict choices; ties can produce multiple SPNE.

3.4 Restrictions of a strategy profile

Meaning: to test a subgame, keep only the actions prescribed at information sets inside it. **Why:** equilibrium is checked within the continuation. **Example:** after Entry, inspect the incumbent's response strategy.

Interview: do not let earlier sunk actions influence the continuation comparison except through payoffs/state.

3.5 SPNE with stage games

Meaning: a subgame can contain simultaneous decisions, so solve its Nash equilibria rather than select one action at a node. **Why:** backward induction generalizes to equilibrium correspondences. **Example:** pricing simultaneously after investment. **Interview:** multiple continuation equilibria may support multiple SPNE.

4. Real-World Example

A service chooses whether to adopt a new authentication protocol. If it adopts, two client teams simultaneously choose whether to migrate. The adoption continuation is a coordination subgame: both migrating and both staying may be Nash equilibria with different payoffs. To find SPNE, first enumerate Nash equilibria of that migration subgame, then evaluate the service's adoption choice under each continuation equilibrium. This captures a common rollout problem: a platform decision depends on which stable client-coordination outcome follows.

5. Diagrams / Mental Models

SPNE profile

- | Nash equilibrium in whole game
- | Nash equilibrium in proper subgame A
- | Nash equilibrium in nested subgame A.1
- | Nash equilibrium in every off-path proper subgame

Test	Nash	SPNE
Whole-game unilateral deviations	Yes	Yes
Equilibrium in every proper subgame	No	Yes
Eliminates exposed empty threats	Not necessarily	Yes
Defined for normal-form game alone	Yes	Needs extensive-form/subgames

6. Common Interview Questions

1. **Define SPNE.** A strategy profile inducing a Nash equilibrium in every subgame. **Expected:** includes whole and off-path subgames. **Mistake:** "a Nash equilibrium found backward."
2. **Is every SPNE a Nash equilibrium?** Yes, because the entire game is a subgame. **Expected:** short proof. **Mistake:** reversing the implication.
3. **Is every Nash equilibrium SPNE?** No; Nash may use non-credible off-path threats. **Expected:** entry example. **Mistake:** saying never.

4. **How do you find SPNE in finite perfect-information games?** Backward induction, retaining all optimal actions when tied. **Expected:** full strategies. **Mistake:** reporting only path.
5. **Why does SPNE eliminate empty threats?** A suboptimal threat fails Nash equilibrium in the continuation subgame where it would be executed. **Expected:** local deviation. **Mistake:** appeal only to intuition.
6. **Can there be multiple SPNE?** Yes, from tied choices or multiple Nash equilibria in continuation subgames. **Expected:** multiplicity. **Mistake:** assuming backward induction is always unique.
7. **Can a game have Nash equilibria but no pure SPNE?** Yes when a proper subgame has no pure Nash equilibrium; a mixed SPNE may exist. **Expected:** stage-game distinction. **Mistake:** claiming finite always means pure SPNE without perfect information.
8. **What if there are no proper subgames?** SPNE and Nash impose the same condition. **Expected:** imperfect-information example. **Mistake:** claiming SPNE is always strictly stronger.
9. **Does SPNE require optimality at every node?** In perfect-information games effectively yes; generally it requires Nash in every subgame, which may not start at every information set. **Expected:** qualification. **Mistake:** overclaiming under imperfect information.
10. **SPNE versus backward induction?** SPNE is an equilibrium concept; backward induction is a solution procedure that produces SPNE under suitable finite perfect-information assumptions. **Expected:** concept versus algorithm. **Mistake:** treating them as synonyms.

7. Deep-Dive Questions

1. **Why does one-deviation reasoning work in finite perfect-recall games?** If a profitable multi-stage deviation exists, consider its last differing decision; improving there yields a profitable one-stage deviation, contradicting sequential optimality.
2. **How do mixed strategies enter SPNE?** The restriction to every subgame must be a possibly mixed Nash equilibrium; behavioral strategies randomize at information sets.
3. **Why might SPNE be too weak?** If imperfect information prevents proper subgames, implausible beliefs or actions may remain. Sequential equilibrium and perfect Bayesian equilibrium refine further.
4. **Can commitment alter the SPNE rather than merely select it?** Yes. Removing an action or changing payoffs changes the extensive-form game and therefore its subgames and equilibria.
5. **How do multiple continuation equilibria affect the root?** Each equilibrium continuation supplies potentially different payoffs to earlier players, so equilibrium selection downstream can change whether earlier actions are optimal.

8. Comparison Tables

Concept	Stability requirement	Handles off-path continuations?	Typical tool
Nash equilibrium	Whole strategy profile	Weakly	Best responses
SPNE	Every subgame	Yes, when a subgame exposes them	Backward induction
Sequential equilibrium	Every information set plus consistent beliefs	More fully	Assessments and belief consistency

Property	Backward induction	SPNE
Type	Procedure	Equilibrium refinement
Input	Finite extensive-form structure	Extensive-form game and strategy profile
Output/test	Constructs strategies	Classifies profiles
Exact relation	Produces SPNE in finite perfect-information games	May require solving stage-game Nash equilibria more generally

9. Common Mistakes

- Reversing “SPNE implies Nash.”
- Checking only subgames on the equilibrium path.
- Treating actions as complete strategies.
- Assuming finite games always have a pure SPNE without perfect information.
- Ignoring multiple Nash equilibria in a continuation stage.
- Saying SPNE guarantees social efficiency or fairness.
- Assuming SPNE fully handles beliefs at non-singleton information sets.

10. Edge Cases / Special Cases

- No proper subgames: SPNE equals Nash.
- Ties: weakly optimal actions can generate several SPNE.
- Simultaneous continuation: solve a Nash equilibrium, possibly mixed.
- Imperfect information: SPNE may leave implausible off-path beliefs unrestricted.
- Infinite horizon: existence and construction may require continuity, discounting, stationary strategies, or fixed-point arguments.
- A unique equilibrium path can coexist with multiple SPNE that differ only off path.

11. How to Explain in Interview

“SPNE is a Nash equilibrium that remains Nash in every subgame, including off-path continuations. Therefore every SPNE is Nash, but Nash equilibria supported by non-credible threats may fail SPNE. In a finite perfect-information game I find it by backward induction and report complete contingent strategies; ties may yield several SPNE.”

12. Quick Revision Notes

- Definition: Nash in every subgame.
- Inclusion: **SPNE subset of Nash**.
- Purpose: sequential rationality and credible continuation play.
- Procedure: backward induction under finite perfect information.
- Traps: whole strategies, off-path subgames, multiplicity, imperfect information.

13. Practice Tasks

1. Write the strategic form of the entry game and identify Nash versus SPNE profiles.
2. Create a tree with two tied final actions and enumerate all SPNE.

3. Add a simultaneous coordination subgame and solve by equilibrium correspondence.
4. Draw a game with no proper subgames and explain why SPNE adds nothing.
5. Implement a small tree solver and make it return all tied backward-induction profiles.

14. Final Cheat Sheet

Item	Recall
Core definition	Nash equilibrium in every subgame
Why it matters	Requires rational behavior after every subgame history
Most asked	SPNE implies Nash, empty threats, backward induction, ties
Main comparison	Nash checks global deviations; SPNE also checks every continuation subgame
One-line answer	"SPNE is Nash everywhere in the game tree, not just from the root."